

Student Accommodation Finder API Test Cases

Postman test procedures - GET and POST only

34 test cases across 6 API areas

Prepared 2026-10-09 from the repository Postman v2.1 collection.

Scope

This guide documents request setup, roles and prerequisites, example payloads, and expected outcomes. It does not execute requests or create database records.

Use a development database. Several POST cases create persistent data. Campus and listing operations may call public geocoding/routing services subject to provider availability and usage limits. Avoid rapid retries.

How to use this guide

1. Start the backend and set `baseUrl` (default `http://localhost:5000`).
2. Send requests from `postman/student-accommodation-finder.postman_collection.json`; Postman tests are attached to each request.
3. Register a disposable student to populate `studentToken`. Supply local admin and verified-landlord credentials, then run the login requests.
4. Set or obtain the required `institutionId`, `campusId`, `listingId`, and `conversationId` for dependent cases.
5. Use only development credentials and data. Provider downtime may return the documented service error.

Collection variables

Variable	Setup
<code>baseUrl</code>	<code>http://localhost:5000</code>
<code>adminEmail</code>	Set in Postman before use
<code>adminPassword</code>	Set in Postman before use
<code>landlordEmail</code>	Set in Postman before use
<code>landlordPassword</code>	Set in Postman before use
<code>studentToken</code>	Set in Postman before use
<code>adminToken</code>	Set in Postman before use
<code>landlordToken</code>	Set in Postman before use
<code>studentId</code>	Set in Postman before use
<code>institutionId</code>	Set in Postman before use
<code>campusId</code>	Set in Postman before use
<code>listingId</code>	Set in Postman before use
<code>conversationId</code>	Set in Postman before use

Safety: Do not use production credentials or run persistent-write cases against shared production data.

Contents

1. Health
2. Authentication
3. Campuses and institutions
4. Listings and student interactions
5. Landlord
6. Admin reads

1. Health

TC-01

GET

GET API health

Endpoint: `{{baseUrl}}/api/health`

Purpose, prerequisites and expected result

Public liveness check. Expected: HTTP 200 and success=true.

Authentication: None (public endpoint)

Request data

No request body.

Postman checks

- HTTP 200
- success envelope

2. Authentication

Register a disposable student before running student-only calls. Admin and landlord credentials must be supplied as local Postman variables.

TC-02 POST POST Register test student

Endpoint: `{{baseUrl}}/api/auth/register`

Purpose, prerequisites and expected result

Creates a unique disposable student account. Expected: HTTP 201; automatically saves the returned student token and ID.

Authentication: None (public endpoint)

Request data

```
{
  "name": "Postman Test Student {{$guid}}",
  "email": "postman-student-{{$guid}}@example.com",
  "password": "PostmanTestPass123!",
  "role": "STUDENT"
}
```

Postman checks

- HTTP 201
- success envelope and student token

TC-03 POST POST Login admin

Endpoint: `{{baseUrl}}/api/auth/login`

Purpose, prerequisites and expected result

Set adminEmail and adminPassword in the local Postman environment. Expected: HTTP 200; saves adminToken.

Authentication: None (public endpoint)

Request data

```
{
  "email": "{{adminEmail}}",
  "password": "{{adminPassword}}"
}
```

Postman checks

- HTTP 200
- admin login succeeds

TC-04

POST

POST Login verified landlord**Endpoint:** `{{baseUrl}}/api/auth/login`**Purpose, prerequisites and expected result**

Set credentials for an active, verified landlord in the local Postman environment. Needed for landlord-only listing calls.

Authentication: None (public endpoint)**Request data**

```
{
  "email": "{{landlordEmail}}",
  "password": "{{landlordPassword}}"
}
```

Postman checks

- HTTP 200
- verified landlord login succeeds

TC-05

POST

POST Reject public admin registration**Endpoint:** `{{baseUrl}}/api/auth/register`**Purpose, prerequisites and expected result**

Negative authorization/business-rule test. Expected: HTTP 400; public callers cannot register admin accounts.

Authentication: None (public endpoint)**Request data**

```
{
  "name": "Should Not Become Admin",
  "email": "public-admin-{{$guid}}@example.com",
  "password": "PostmanTestPass123!",
  "role": "ADMIN"
}
```

Postman checks

- HTTP 400
- registration rejected

TC-06

GET

GET Current authenticated user

Endpoint: `{{baseUrl}}/api/auth/me`

Purpose, prerequisites and expected result

Requires studentToken from Register test student. Expected: HTTP 200 with the authenticated user.

Authentication: Bearer `{{studentToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- returns authenticated user

TC-07

GET

GET Current profile

Endpoint: `{{baseUrl}}/api/users/me`

Purpose, prerequisites and expected result

Requires studentToken. Expected: HTTP 200; returned profile must not contain a password hash.

Authentication: Bearer `{{studentToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- profile excludes password hash

3. Campuses and institutions

TC-08

GET

GET Active institutions

Endpoint: `{{baseUrl}}/api/campuses/institutions`

Purpose, prerequisites and expected result

Public read. Expected: HTTP 200 and an array of active institutions.

Authentication: None (public endpoint)

Request data

No request body.

Postman checks

- HTTP 200
- data is an array

TC-09

GET

GET Active campuses

Endpoint: `{{baseUrl}}/api/campuses`

Purpose, prerequisites and expected result

Public read. Expected: HTTP 200 and an array of active campuses.

Authentication: None (public endpoint)

Request data

No request body.

Postman checks

- HTTP 200
- data is an array

TC-10

GET

GET Institution suggestions (admin)**Endpoint:** `{{baseUrl}}/api/campuses/institutions/suggestions?search=tshwane`**Purpose, prerequisites and expected result**

Requires adminToken. Calls the OpenAlex provider. Expected: HTTP 200, or a documented 503/502 if the provider is unavailable.

Authentication: Bearer `{{adminToken}}`**Request data**

No request body.

Postman checks

- documented response status
- suggestions are an array

TC-11

GET

GET Campus suggestions for institution (admin)**Endpoint:** `{{baseUrl}}/api/campuses/suggestions?institutionId={{institutionId}}`**Purpose, prerequisites and expected result**

Set institutionId to an active institution UUID. Requires adminToken and calls OpenStreetMap Overpass, which is rate limited. Expected: HTTP 200 with campus suggestions, or a documented provider error.

Authentication: Bearer `{{adminToken}}`**Request data**

No request body.

Postman checks

- documented response status
- suggestions are an array

TC-12

POST

POST Create institution (admin; persists data)**Endpoint:** `{{baseUrl}}/api/campuses/institutions`**Purpose, prerequisites and expected result**

Admin-only write; creates a persistent institution. Use a disposable database. Expected: HTTP 201; saves institutionId.

Authentication: Bearer `{{adminToken}}`**Request data**

```
{
  "name": "Postman Test Institution {{guid}}",
  "shortName": "PM{{guid}}"
}
```

Postman checks

- HTTP 201
- institution created

TC-13

POST

POST Create campus (admin; geocodes address)**Endpoint:** `{{baseUrl}}/api/campuses`**Purpose, prerequisites and expected result**

Admin-only persistent write; requires institutionId. Resolves the address through rate-limited OpenStreetMap. Use a disposable database and do not retry rapidly. Expected: HTTP 201; saves campusId.

Authentication: Bearer `{{adminToken}}`**Request data**

```
{
  "name": "Postman Test Campus",
  "institutionId": "{{institutionId}}",
  "location": "Pretoria, Gauteng",
  "address": "2 Aubrey Matlala Road, Soshanguve, Pretoria, South Africa"
}
```

Postman checks

- campus created or geocoder reports a documented failure
- campus has coordinates

4. Listings and student interactions

Set listingId to an approved listing UUID before running student interaction requests. Report, favourite, review, and enquiry requests create persistent records.

TC-14

GET

GET Search public listings

Endpoint: `{{baseUrl}}/api/listings?search=student&sort=newest&page=1&limit=12`

Purpose, prerequisites and expected result

Public search. Expected: HTTP 200, paginated results contain approved listings only.

Authentication: None (public endpoint)

Request data

No request body.

Postman checks

- HTTP 200
- search response includes results

TC-15

GET

GET Listing detail

Endpoint: `{{baseUrl}}/api/listings/{{listingId}}`

Purpose, prerequisites and expected result

Set listingId to an approved listing. Expected: HTTP 200 and listing details.

Authentication: None (public endpoint)

Request data

No request body.

Postman checks

- HTTP 200
- approved listing returned

TC-16

GET

GET Listing reviews**Endpoint:** `{{baseUrl}}/api/listings/{{listingId}}/reviews`**Purpose, prerequisites and expected result**

Set listingId to an approved listing. Expected: HTTP 200 and review array.

Authentication: None (public endpoint)**Request data**

No request body.

Postman checks

- HTTP 200
- data is an array

TC-17

POST

POST Submit review (student; one review per listing)**Endpoint:** `{{baseUrl}}/api/listings/{{listingId}}/reviews`**Purpose, prerequisites and expected result**

Requires studentToken and an approved listing not previously reviewed by this student. Optional comment is empty. Expected: HTTP 201; a repeat for the same student/listing returns 409.

Authentication: Bearer `{{studentToken}}`**Request data**

```
{
  "rating": 5,
  "comment": ""
}
```

Postman checks

- review created or already submitted
- rating-only review accepted

TC-18

POST

POST Submit report (student; persists data)**Endpoint:** `{{baseUrl}}/api/listings/{{listingId}}/reports`**Purpose, prerequisites and expected result**

Requires studentToken and approved listingId. Creates a persistent test report; use only in a disposable database. Expected: HTTP 201 and success message.

Authentication: Bearer `{{studentToken}}`**Request data**

```
{
  "reason": "Postman workflow test",
  "description": "TEST ONLY: verify report submission and student report tracking."
}
```

Postman checks

- HTTP 201
- report submitted

TC-19

GET

GET My reports (student)**Endpoint:** `{{baseUrl}}/api/reports/mine`**Purpose, prerequisites and expected result**

Requires studentToken. Expected: HTTP 200 and only reports submitted by the authenticated student.

Authentication: Bearer `{{studentToken}}`**Request data**

No request body.

Postman checks

- HTTP 200
- reports returned as array

TC-20

POST

POST Save favourite (student)

Endpoint: `{{baseUrl}}/api/users/favourites/{{listingId}}`

Purpose, prerequisites and expected result

Requires `studentToken` and approved `listingId`. Creates a persistent favourite; repeated saves may return a conflict.

Authentication: Bearer `{{studentToken}}`

Request data

No request body.

Postman checks

- favourite added or already exists

TC-21

GET

GET My favourites (student)

Endpoint: `{{baseUrl}}/api/users/favourites`

Purpose, prerequisites and expected result

Requires `studentToken`. Expected: HTTP 200 and favourite array.

Authentication: Bearer `{{studentToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- data is an array

TC-22

POST

POST Start listing conversation (student)**Endpoint:** `{{baseUrl}}/api/conversations`**Purpose, prerequisites and expected result**

Requires studentToken and approved listingId. Creates a conversation or returns the existing one; saves conversationId.

Authentication: Bearer `{{studentToken}}`**Request data**

```
{
  "listingId": "{{listingId}}"
}
```

Postman checks

- HTTP 200 or 201
- conversation returned

TC-23

GET

GET My conversations**Endpoint:** `{{baseUrl}}/api/conversations`**Purpose, prerequisites and expected result**

Requires studentToken. Expected: HTTP 200 and the student's conversations.

Authentication: Bearer `{{studentToken}}`**Request data**

No request body.

Postman checks

- HTTP 200
- data is an array

TC-24

POST

POST Send conversation message (student)**Endpoint:** `{{baseUrl}}/api/conversations/{{conversationId}}/messages`**Purpose, prerequisites and expected result**

Requires studentToken and conversationId from Start listing conversation. Creates a persistent test message.

Authentication: Bearer `{{studentToken}}`**Request data**

```
{
  "content": "TEST ONLY: Postman student-to-landlord messaging check."
}
```

Postman checks

- HTTP 201
- message persisted

TC-25

GET

GET Conversation messages**Endpoint:** `{{baseUrl}}/api/conversations/{{conversationId}}/messages`**Purpose, prerequisites and expected result**

Requires studentToken belonging to the conversation. Expected: HTTP 200 and ordered message array.

Authentication: Bearer `{{studentToken}}`**Request data**

No request body.

Postman checks

- HTTP 200
- messages returned as array

TC-26

GET

GET My notifications

Endpoint: `{{baseUrl}}/api/notifications`

Purpose, prerequisites and expected result

Requires studentToken. Expected: HTTP 200 and the authenticated user's notifications.

Authentication: Bearer `{{studentToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- notifications returned as array

5. Landlord

TC-27

GET

GET My listings (landlord)

Endpoint: `{{baseUrl}}/api/listings/my`

Purpose, prerequisites and expected result

Requires landlordToken from the verified landlord login. Expected: HTTP 200 and only that landlord's listings.

Authentication: Bearer `{{landlordToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- listings returned as array

TC-28

POST

POST Create listing (verified landlord; persists and geocodes)**Endpoint:** `{{baseUrl}}/api/listings`**Purpose, prerequisites and expected result**

Requires an active verified landlord and campusId. Persistent write; geocodes the property and campus via rate-limited OpenStreetMap/OSRM. Use a disposable database and a near-campus valid address. Add a photos form-data file only if desired. Expected: HTTP 201 and PENDING approval status.

Authentication: Bearer `{{landlordToken}}`**Request data**

Form field	Value
title	Postman Test Student Room
description	TEST ONLY: temporary API validation listing.
campusId	{{campusId}}
accommodationType	ROOM
pricePerMonth	4000
address	Molotlegi Road, Ga-Rankuwa, Gauteng, South Africa
totalRooms	4
availableRooms	1
amenities	["Wi-Fi","Security"]
availabilityStatus	AVAILABLE

Postman checks

- listing created or documented geocoder failure
- listing awaits moderation

6. Admin reads

All requests require adminToken from Login admin.

TC-29

GET

GET Pending listings

Endpoint: `{{baseUrl}}/api/admin/listings/pending`

Purpose, prerequisites and expected result

Expected: HTTP 200 and pending listing array.

Authentication: Bearer `{{adminToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- all returned listings are pending

TC-30

GET

GET Admin listings

Endpoint: `{{baseUrl}}/api/admin/listings?status=ALL`

Purpose, prerequisites and expected result

Expected: HTTP 200 and all listings. Supported statuses: ALL, DRAFT, PENDING, APPROVED, REJECTED.

Authentication: Bearer `{{adminToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- data is an array

TC-31

GET

GET Admin statistics

Endpoint: `{{baseUrl}}/api/admin/stats`

Purpose, prerequisites and expected result

Expected: HTTP 200 and aggregate platform counts.

Authentication: Bearer `{{adminToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- stats include listing counts

TC-32

GET

GET Admin reports

Endpoint: `{{baseUrl}}/api/admin/reports`

Purpose, prerequisites and expected result

Expected: HTTP 200 and report array for moderation.

Authentication: Bearer `{{adminToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- data is an array

TC-33

GET

GET Admin users

Endpoint: `{{baseUrl}}/api/admin/users?role=LANDLORD`

Purpose, prerequisites and expected result

Expected: HTTP 200 and landlord directory; password hashes must not be included.

Authentication: Bearer `{{adminToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- user records exclude password hashes

TC-34

GET

GET Admin audit logs

Endpoint: `{{baseUrl}}/api/admin/audit-logs`

Purpose, prerequisites and expected result

Expected: HTTP 200 and audit log array.

Authentication: Bearer `{{adminToken}}`

Request data

No request body.

Postman checks

- HTTP 200
- audit logs returned as array